Resumen de Estudio — Diseño de Bases de Datos II (2054)

Ingeniería en Informática — UM

Nota sobre las fuentes utilizadas

Este resumen sigue exactamente el Programa Analítico de la materia. Para su elaboración usé estos tipos de fuente:

1. **Diapositivas de cátedra:**
    - **Ing. Carina Cozzolino** (Profesora Titular Adjunta): *Inteligencia de Negocios*, *Bases de Datos Móviles*, *Otras Arquitecturas y Plataformas*, *Bases de Datos Corporativas*, *Proceso ETL*.
    - **Prof. Ing. Fernando Zapata** (Profesor Titular): *Intro Infraestructura*, *Clases GIS*, *Intro DB NoSQL*, *Intro MongoDB*, *MongoDB Instalación y Configuración*, *MongoDB Modelado*, *MongoDB Índices y Rendimiento*, *MongoDB Operación — Gestión de Datos*, *MongoDB Operación — Backup*, *MongoDB Cloud*.

    ⚠️ Las presentaciones del Prof. Zapata incluyen una nota explícita: *"Esta presentación constituye sólo una guía del docente para el dictado de la clase. No debe considerarse como material de estudio para los estudiantes, ya que los temas contenidos se encuentran sólo mencionados y carecen de explicaciones, definiciones y ejemplos de aplicación práctica."* Por eso, en este resumen tomé esos contenidos (que suelen ser muy densos: comandos, sintaxis exacta, listas de conceptos) y los reconstruí con explicaciones completas y ejemplos — pero la terminología, el orden y los ejemplos técnicos puntuales (comandos de MongoDB, tablas, casos reales) son fieles a lo que muestra la cátedra, así que reflejan lo que probablemente se evalúe.

    Es la fuente más importante en ambos casos porque es literalmente lo que tus profesores explican en clase. Marcada con 🔵📘.

2. **El trabajo de titulación** *"Análisis, diseño, construcción e implementación de un geoportal con las herramientas MongoDB, JSON y GeoJSON"* (Coba & Gómez, UPS, 2014), que cita textualmente a **Kristina Chodorow** (autora de *MongoDB: The Definitive Guide*, bibliografía de la Unidad 3, confirmada también en la diapositiva de bibliografía

del Prof. Zapata). Marcada con 🔵.

3. **Conocimiento técnico general del área**, usado solo donde ni el programa ni las diapositivas de cátedra cubren el detalle (por ejemplo, los libros de Mattison/Anahory sobre Data Warehousing específicamente, o el manual de Murillo Alfaro, cuyo texto no me fue proporcionado). Marcada con 🟡. A esta altura queda muy poco contenido con esta marca: principalmente el modelado dimensional del Tema 3 de la Unidad 2 (esquema estrella/copo de nieve como concepto de Data Warehousing "puro") y el detalle de Procesamiento Analítico (OLAP) y Minería de Datos (Unidad 2, Temas 4 y 5), de los cuales no recibí diapositivas de cátedra todavía.

UNIDAD 1: SISTEMAS DE BASES DE DATOS MODERNOS

Tema 1: Introducción a los sistemas de bases de datos modernos 🔵📘 (fuente: diapositivas de cátedra, Prof. Ing. Fernando Zapata — *"Intro Infraestructura"*)

> 📌 Esta presentación del profesor trae la aclaración de que es *"sólo una guía del docente para el dictado de la clase"* y no reemplaza la bibliografía. Aun así, conserva la terminología y estructura exactas que usa la cátedra, muy útil para el examen.

1.1 Contexto, escenarios y necesidades. Componentes y procesos de datos en sistemas de información modernos

De las apps monolíticas a las modernas: la cátedra parte de comparar el desarrollo "de antes" (codificación → compilación → publicación, una app monolítica pensada para un solo dispositivo) con el desarrollo actual, donde una aplicación debe funcionar en **múltiples dispositivos con distintas tecnologías y formatos**, con **acceso a través de internet** y como **aplicaciones distribuidas**.

Arquitectura en capas — la cátedra explica el reparto de responsabilidades en tres capas:

- **CP (Capa de Presentación):** todo lo que el usuario ve — formularios, menús, reportes — y validaciones simples que no requieren acceso a la base de datos (ej. que la fecha de venta no sea posterior a hoy).
- **CLN (Capa de Lógica del Negocio):** las validaciones complejas, las entidades y los procesos (ej. que no se le venda a crédito a un cliente moroso). Cada entidad

(Empleado, Cliente, etc.) tiene aquí sus rutinas: grabar, obtener por ID, obtener todos, etc.

- **CAD (Capa de Acceso a Datos):** los comandos que se comunican directamente con la base de datos (conectar, desconectar, iniciar/finalizar transacción, ejecutar procedimiento almacenado). La clave de este diseño es que **la aplicación no depende del origen de los datos**: se puede usar tablas .DBF o cualquier motor SQL (Firebird, MySQL, MariaDB, PostgreSQL, SQL Server) sin que la CP ni la CLN se enteren del cambio — solo hay que reescribir los comandos específicos en la CAD.

Technology Stack (Stack Tecnológico): según la cátedra, es *"una lista de todos los servicios tecnológicos, sistemas, herramientas y componentes utilizados para crear y ejecutar una aplicación"*, incluyendo lenguajes, bases de datos, bibliotecas, frameworks y herramientas de frontend y backend.

Stacks tecnológicos comúnmente usados (según cátedra):

- **LAMP** (Linux, Apache, MySQL, PHP): stack seguro, flexible y con buenas opciones de personalización.
- **Django** (Python): framework de alto nivel, código abierto y gratuito. Permite rápida implementación, ideal para crear MVP (productos mínimos viables).
- **MEAN** (MongoDB, Express.js, Angular, Node.js): usado principalmente en aplicaciones de alto rendimiento y aplicaciones web escalables.
- **MERN** (MongoDB, Express.js, React, Node.js): ha cobrado relevancia con la utilización de React.

Front-End mencionado por cátedra: jQuery, Angular, React, Ember JS. Back-End: Java (Spring/Struts/Hibernate), PHP (Laravel), Python (Django/Flask), Ruby (Ruby on Rails).

Data Stack: la cátedra referencia el famoso mapa *"The 2021 Machine Learning, AI and Data (MAD) Landscape"* (Matt Turck) para mostrar la enorme cantidad de herramientas del ecosistema de datos actual (más allá del desarrollo de aplicaciones tradicional).

1.2 Bases de datos en la nube. Evolución y tendencias. Arquitecturas. Principales actores

Clustering: la cátedra lo define como *sistemas distribuidos formados por "granjas" de computadoras unidas entre sí, normalmente por una red de alta velocidad, que se comportan como si fuesen un único servidor*. Menciona el concepto de **HPC** (High Performance Computer / cluster computer), con ejemplos históricos reales (cluster construido con PCs de uso común en 1995; cluster HPC "Minerva" del Instituto Max Planck en 2016).

Virtualización: definida como la *capacidad de que múltiples máquinas con sus aplicaciones puedan ejecutarse dentro de un mismo equipo físico*, mediante un **Hypervisor** (que gestiona los recursos: procesamiento, almacenamiento, red). Beneficios que da la cátedra: reducción de costos, reducción de gastos operativos (UPS, administradores, energía, espacio), optimización de recursos, menor latencia entre servidores, aislamiento y flexibilidad.

Cloud Computing: definido por la cátedra como un *"conjunto de recursos informáticos virtualizados y compartidos, para ser utilizados bajo demanda"*, donde los recursos pueden ser de tipo cómputo, almacenamiento o red. Se organiza en:

- **Public / Private Cloud**
- **SaaS** (Software as a Service — ej. Visual Studio, SharePoint, SQL Server como servicio)
- **PaaS** (Platform as a Service)
- **IaaS** (Infrastructure as a Service — CPU, memoria, almacenamiento)
- Y se apoya en Clustering y Georreplicación.

Big Data (concepto transversal que la cátedra ubica también en esta introducción)

La cátedra define **Big Data** como *el manejo y análisis de grandes volúmenes de datos que son demasiado complejos, grandes y diversos para ser procesados con las herramientas tradicionales*, provenientes de fuentes como redes sociales, transacciones comerciales, sensores o dispositivos móviles. Las **5 V** que hay que considerar, según cátedra:

- **Volumen**: cantidad masiva de datos generados continuamente.
- **Velocidad**: rapidez con la que se generan y procesan.
- **Variedad**: distintos tipos de datos (estructurados, no estructurados, semiestructurados).
- **Veracidad**: calidad y precisión de los datos.
- **Valor**: utilidad que se puede extraer de ellos.

Usos de Big Data (según cátedra): en negocios (personalización de productos, marketing dirigido), en salud (diagnóstico médico, investigación genética, monitoreo de epidemias), en gobierno (políticas públicas basadas en datos, prevención de fraude) y en ciencia (investigación, análisis de grandes datasets experimentales). Big Data se apoya en tecnologías como **Hadoop**, **Spark** y bases de datos **NoSQL**.

Ejemplo aplicado a la vida cotidiana: cuando usás Netflix, tu experiencia depende de todo

este stack: la app en tu celular (CP) se conecta a través de internet a servicios distribuidos que corren en la nube de Amazon (IaaS/PaaS), sobre clusters virtualizados que escalan automáticamente en los picos de tráfico (viernes a la noche), mientras un pipeline de Big Data (con las 5 V) procesa en tiempo real qué mirás para recomendarte el próximo contenido — todo esto sin que vos notes nada de la infraestructura detrás.

Tema 2: Inteligencia de Negocios 🔵📘 (fuente: diapositivas de cátedra, Ing. Cozzolino)

2.1 Conceptos. Bases de datos corporativas y de apoyo a la toma de decisiones

¿Qué es Business Intelligence? Según la definición de cátedra: es el *conjunto de estrategias, tecnologías y herramientas orientadas a transformar datos en información útil, facilitando la toma de decisiones dentro de una organización*. La idea central que transmite la cátedra es que **BI origina una nueva forma de competir: decisiones basadas en datos objetivos y no en intuición**.

Beneficios de la Inteligencia de Negocios (tal como los enumera la cátedra):

- Genera un círculo virtuoso: **datos → información → conocimiento → mejores decisiones**.
- Brinda una visión única, fiable y consolidada de información relevante.
- Permite la creación y seguimiento de métricas.
- Facilita el acceso a información actualizada en todos los niveles.
- Alinea los intereses entre TI y la organización.

Base de Datos Operativa vs. Base de Datos Corporativa — esta comparación es clave y suele tomarse en examen:

Base de Datos Operativa	Base de Datos Corporativa
Datos operativos	Datos integrados
Normalizados (muchas tablas y pocas columnas)	Desnormalizados (pocas tablas y muchas columnas)
Actualización permanente	Actualización por reglas regulares

Alta, baja, modificación y consulta	Procesado de datos
Informes periódicos	Refleja un modelo de negocios
Pequeños volúmenes de datos	Información bajo demanda
Muchos usuarios	Pocos usuarios
Uso: día a día	Uso: toma de decisiones

Diferencias entre BI, Analítica y Big Data (tabla de cátedra):

	BI (Business Intelligence)	**Analítica (Analytics)**	**Big Data**
Enfoque principal	Datos actuales/pasados	Predicciones de futuro	Volumen, variedad y velocidad
Uso de datos	Descriptivo (histórico y actual)	Predictivo y prescriptivo	Masivo, estructurado y no estructurado
Usuarios	Directivos y analistas generales	Analistas de datos expertos	Toda la organización

2.2 Plataformas más utilizadas. Herramientas y automatización. Licenciamiento

Estrategia y arquitectura de BI: definir una estrategia de BI implica conocer el grado de madurez de la organización (modelo **BIMM**, Business Intelligence Maturity Model), y definir una arquitectura que cubra todo el ciclo de vida del dato, desde el origen hasta la explotación. Los **componentes** de esa arquitectura, según la cátedra, son:

- **Origen de datos**: bases de datos, archivos, aplicaciones, web.
- **ETL**: procesos de extracción, transformación y carga de datos (ver Tema 2 de la Unidad 2).
- **Data Warehouse**: repositorio centralizado y estructurado.
- **Herramientas de explotación**: dashboards, análisis OLAP.

Fases para implementar un proyecto de BI (según cátedra):

1. **Planificación**: identificación de áreas de oportunidad y tecnologías adecuadas.
2. **Análisis de requerimientos**: determinación de necesidades de información.

3. **Selección de arquitectura y modelado**: diseño lógico y arquitectura tecnológica.
4. **Procesos ETL**: extracción, transformación y carga de datos.
5. **Explotación de información**: desarrollo de dashboards y consultas dinámicas.

Herramientas principales de BI (tabla de cátedra):

Herramienta	Característica destacada
Power BI (Microsoft)	Fácil de usar, versión gratuita para escritorio, integración con otras soluciones de Microsoft
Tableau	Potente en visualizaciones y análisis avanzado
Qlik Sense	Destacada por su modelado asociativo de datos
SAP Business Objects / Pentaho / Oracle BI	Soluciones robustas para empresas de mayor porte

Automatización en BI: actualizaciones automáticas de reportes, alertas configurables, análisis en tiempo real.

Licenciamiento: gratuito (ej. Power BI Desktop), comercial por usuario o suscripción, y la disyuntiva Cloud vs. On-premise (cada una con sus ventajas y desafíos).

Ejemplos reales de uso de BI que da la cátedra:

- **Netflix**: usa algoritmos de BI para recomendar contenido personalizado y decidir qué series o películas producir, según el comportamiento de visualización.
- **Amazon**: analiza patrones de compra, búsquedas y devoluciones para personalizar ofertas, prever demanda y optimizar logística.
- **Carrefour**: usa BI para ajustar precios dinámicamente, anticipar la demanda según eventos o clima, y optimizar el inventario por sucursal.
- **Sector salud (hospitales)**: implementan BI para predecir ocupación de camas, administrar recursos críticos y detectar patrones de enfermedades.

2.3 Acceso a datos: nociones de Open Data y protección de datos personales

Open Data (Información Pública): según la cátedra, responde al derecho de información, a la publicidad de los actos de gobierno, y busca garantizar la transparencia y evitar abusos.

Protección de Datos Personales: la cátedra la enmarca en la ley argentina de Protección

de Datos Personales (la norma vigente en Argentina es la **Ley N.º 25.326**), y distingue dos categorías:

- **Datos personales**: información referida a personas físicas o jurídicas.
- **Datos sensibles**: origen racial y étnico, opiniones políticas, convicciones religiosas/filosóficas/morales, afiliación sindical, e información referente a la salud o vida sexual.

Ejemplo aplicado a la vida cotidiana: cuando entrás al sitio de datos abiertos de tu municipio (Open Data) y descargás el padrón de comercios habilitados, estás usando datos públicos sin restricción. Pero si una app de salud guarda tu historial clínico para hacer analítica de BI y predecir riesgos, esos datos son "sensibles" según la Ley 25.326: la empresa tiene que anonimizarlos o pedir tu consentimiento explícito antes de analizarlos, exactamente la misma lógica que separa "Open Data" de "datos sensibles" en el cuadro de la cátedra.

Tema 3: Bases de datos móviles 🔵📘 (fuente: diapositivas de cátedra, Ing. Cozzolino)

Definición y ventajas

La cátedra define las bases de datos móviles como *bases de datos diseñadas para funcionar en dispositivos móviles, permitiendo el almacenamiento y acceso a datos sin importar la conectividad*. Sus ventajas principales son:

- **Movilidad**: datos disponibles donde sea.
- **Sincronización**: acceso online y offline.
- **Variedad** de dispositivos y plataformas.

3.1 Arquitectura informática móvil. Tipos de datos en aplicaciones móviles

La **arquitectura móvil** es la estructura y organización de los componentes de una app móvil (arquitectura cliente-servidor), con tres componentes principales según la cátedra:

- **Cliente móvil**: el dispositivo que ejecuta la app.
- **Middleware**: traduce las peticiones entre cliente y servidor.
- **Servidor**: provee acceso a bases de datos remotas.

Patrones más usados para organizar el código de la app: **MVC** (Modelo-Vista-Controlador,

separa la lógica, la interfaz y el control) y **MVP** (Modelo-Vista-Presentador, mejora la separación de la lógica de presentación).

Tipos de datos que manejan las apps móviles, según la cátedra:

- **Configuración**: preferencias del usuario, estado.
- **Archivos**: imágenes, documentos, audio, multimedia.
- **Datos transaccionales**: registros de operaciones, compras y mensajes.
- **Datos sensibles**: contraseñas, identificadores de dispositivos, ubicaciones.
- **Datos estructurados y no estructurados**: relacionales (tablas) o NoSQL (clave-valor, documentos, grafos, objetos).

Tipo de datos	Ejemplo
Datos locales	Preferencias, configuraciones
Datos temporales	Caché, formularios sin enviar
Datos sincronizados	Usuarios, contenidos
Sensible	Token de autenticación

3.2 Cuestiones de la gestión de datos

La **gestión de datos móviles**, según la cátedra, debe cubrir:

- **Seguridad y privacidad**: protección ante robos y accesos no autorizados.
- **Sincronización de la información**: entre dispositivos y la nube.
- **Consumo de recursos**: batería, almacenamiento, ancho de banda.
- **Escalabilidad y rendimiento**: soporte a grandes volúmenes y múltiples usuarios.
- **Cifrado y respaldo**: proteger datos en reposo y en tránsito, políticas de respaldo automático.
- **Gestión de dispositivos móviles (MDM)**: herramientas para administrar y proteger múltiples dispositivos en una red empresarial.

Tipos de bases de datos usadas en móviles (tabla de cátedra):

- **Relacionales**: basadas en tablas y relaciones (SQL); dan transacciones, integridad y consultas fáciles.

- **No relacionales**:
  - **Documentales** (Firebase, Couchbase, PouchDB): usan documentos JSON/BSON.
  - **Clave-valor** (Redis): datos simples y de acceso rápido.
  - **Grafos** (Neo4j): relaciones complejas.
  - **Orientadas a objetos** (Realm): almacenan directamente objetos.

3.3 Bases de datos sincronizadas intermitentemente

La cátedra define la **sincronización intermitente** con tres ideas clave:

- **Permiten trabajar offline**: el usuario accede y modifica datos aun sin internet.
- **Sincronización**: cuando hay conexión, los cambios se replican con el servidor.
- **Modelos de sincronización**: **incremental** (solo lo que cambió) o **completa** (todo el set de datos).

Riesgos comunes: inconsistencias temporales y conflictos de cambios simultáneos.

Ejemplos que da la cátedra: WhatsApp (mensajes offline) y aplicaciones de notas en la nube.

Tabla de ventajas y desafíos de la conexión intermitente:

Ventajas	Desafíos
Disponibilidad continua	Gestión de conflictos e integridad de datos
Mejor experiencia de usuario en zonas con baja conectividad	Balance entre rendimiento y consistencia

Ejemplo aplicado a la vida cotidiana: cuando estás en el subte sin señal y escribís un mensaje en WhatsApp, la app lo guarda en tu base local del celular con un ícono de "reloj" (pendiente de enviar). Apenas volvés a tener señal, WhatsApp sincroniza automáticamente ese mensaje con el servidor — es exactamente el modelo de sincronización intermitente que describe la cátedra, incluyendo el riesgo de conflicto si, por ejemplo, borraste el chat desde otro dispositivo mientras el mensaje seguía pendiente.

Tema 4: Sistemas de información geográfica (SIG / GIS) 🔵📘

(fuente: diapositivas de cátedra, Prof. Ing. Fernando Zapata — *"Clases GIS"* — y la tesis del geoportal 🔵)

4.1 y 4.2 Campo de aplicación y funcionalidades específicas de los sistemas GIS

Definición de cátedra: *un Sistema de Información Geográfica es un conjunto de herramientas informáticas que permiten la organización, almacenamiento, manipulación, análisis y modelado de datos procedentes del mundo real, que están vinculados a una referencia espacial.*

La cátedra resume la esencia de un GIS en dos preguntas: **QUÉ** (datos de interés) y **DÓNDE** (datos espaciales) — todo dato geográfico combina un atributo temático con una ubicación.

Representación de la realidad: para poder trabajar con el mundo real en una computadora hace falta simplificarlo mediante: **escala** (nivel de detalle/reducción), **mapas y planos** (representaciones 2D), y **representación 3D**.

Proyecciones (contenido central de la cátedra, muy probable de examen): una **proyección** es una *transformación matemática que permite representar la superficie curva de la Tierra (tridimensional) en una superficie plana* (mapa, pantalla, papel). Existen varios tipos, según cátedra:

- Cónica (tangente o secante)
- Orientaciones cilíndricas
- Orientaciones planares (acimutales o cenitales — la superficie de proyección es tangente o secante al globo, y la escala aumenta a medida que uno se aleja del centro)
- Perspectivas de orientación polar
- Ejemplos concretos: **Gauss-Krüger**, **Lambert**, **Estereográfica Polar**, **Transversa de Mercator**, **WGS 1984 Web Mercator**.

La cátedra menciona específicamente el **Sistema de Proyección UPS** (Universal Stereographic Polar), donde la escala aumenta a medida que nos alejamos del centro.

Datum: es un *modelo matemático de la forma de la Tierra* — define el tamaño y forma del elipsoide (o geoide) y su posición respecto al centro de la Tierra, sirviendo de base de referencia para ubicar puntos. Ejemplos que da la cátedra: **WGS84** (usado por GPS), **NAD83** (Norteamérica), **POSGAR** (Argentina). Importante: *el datum por sí solo no es un sistema de coordenadas*, es la "referencia" sobre la cual se construyen los sistemas.

Sistemas de coordenadas — distinción clave de cátedra:

- **Sistema de Coordenadas Geográficas (GCS):** sí incluye un datum; usa **latitud y longitud** para ubicar posiciones sobre ese datum. Ejemplo: WGS84 (lat/long).
- **Sistema de Coordenadas Proyectadas (PCS):** parte de un GCS (con su datum) y le aplica una proyección cartográfica para pasarlo a un plano, usando coordenadas en metros o pies. Ejemplo: UTM Zona 19S con WGS84 (EPSG:32719).

Coordenadas geográficas: compuestas por **latitud** (ángulo entre el plano ecuatorial y la línea que pasa por el punto y el centro de la Tierra — el Ecuador es 0°, el polo norte 90°N), **longitud** (ángulo respecto al meridiano de referencia de Greenwich) y **altitud**. Se pueden expresar en:

- **Sistema sexagesimal**: ej. la provincia de Mendoza está entre los 32° y 37°35' de latitud sur y los 66°30' y 70°35' de longitud oeste (según el IGM).
- **Sistema decimal**: ej. las coordenadas de la sede central de la Universidad de Mendoza son -32.890975, -68.861904.

Sistema UTM (Universal Transversal de Mercator): divide el mundo en **60 husos y 20 bandas**, y expresa las magnitudes en **metros** (a diferencia del sistema geográfico, que usa grados).

4.3 Ejemplos de aplicación y soluciones

La cátedra muestra un proyecto real como ejemplo de aplicación: **Áreas de veda en el Mar Argentino y Zona Común de Pesca Argentino-Uruguaya**. La tesis universitaria que revisé (sobre geoportales con MongoDB) documenta otro caso real: geoportales institucionales (como el de la UPS-Ecuador) que georreferencian obras e instituciones con puntos y polígonos.

4.4 Formatos y plataformas comerciales mayormente utilizadas

Formatos que debe poder almacenar una GeoDatabase, según cátedra:

- **Formato vectorial** — almacena puntos, segmentos y polígonos; ocupa poco espacio y facilita operaciones de filtrado, superposición y cálculos matemáticos (ej. rutas, ciudades, ubicación puntual de elementos).
- **Formato raster** — almacena píxeles, generalmente imágenes fotográficas (aéreas o satelitales); hay que considerar resolución, relación de aspecto y manejo de color; ocupa más espacio que el vectorial.

La creación y mantenimiento de cartografía digital involucra procesos manuales, asistencia de ajuste y corrección, actualización, y **correcciones topológicas**.

Plataformas y software que menciona la cátedra:

- **QGIS**: sistema de información geográfica de software libre y código abierto (Linux, Unix, Mac OS, Windows, Android), uno de los primeros proyectos de la Fundación OSGeo.
- **ArcGIS** (ESRI): conjunto comercial de productos para captura, edición, análisis, diseño, publicación e impresión de información geográfica, con familias como ArcGIS Pro, ArcGIS Online, ArcGIS Server (publicación y gestión web) y ArcGIS Mobile (captura en campo).

> 🔗 Conexión con la Unidad 3: además de los SIG "tradicionales" (basados en bases relacionales como PostGIS/PostgreSQL), la tesis sobre geoportales que revisé muestra una alternativa **no relacional**: guardar los datos espaciales directamente en **MongoDB** usando el formato **GeoJSON**, ya que MongoDB soporta de forma nativa índices geoespaciales. Lo vas a ver en detalle en el Tema 2 de la Unidad 3.

Ejemplo aplicado a la vida cotidiana: cuando tu GPS te ubica en Mendoza, en realidad está trabajando con capas de este sistema completo: usa el **datum WGS84** (el mismo que usan los satélites GPS), calcula tu posición en un **Sistema de Coordenadas Geográficas** (lat/long, ej. -32.89, -68.86 en sistema decimal — como las coordenadas de la sede de la UM que menciona la cátedra), y cuando la app te muestra en un mapa plano, aplica una **proyección** (probablemente Web Mercator, la misma que usa Google Maps) para pasar esa posición esférica a la pantalla plana de tu celular — exactamente el mismo proceso GCS → proyección → PCS que explica la cátedra.

Tema 5: Otras arquitecturas y plataformas 🔵📘 (fuente: diapositivas de cátedra, Ing. Cozzolino)

5.1 Bases de datos orientadas a objetos

La cátedra las define como *bases de datos que manipulan información como objetos (instancias de clases), no solo registros en tablas*, permitiendo almacenar estructuras complejas y relaciones, ideales para aplicaciones científicas, ingeniería o sistemas GIS.

Ventajas clave (según cátedra): manejo nativo de datos complejos, mayor flexibilidad para modelar el mundo real, y soporte para herencia, polimorfismo y encapsulamiento.

Modelado de Datos Orientado a Objetos — conceptos base que da la cátedra:

- **Objeto**: entidad con sus propios atributos y métodos.
- **Clase**: conjunto de objetos con características comunes.
- **Identidad**: cada objeto tiene un identificador único (OID).

- **Herencia**: las subclases pueden ampliar la funcionalidad de las superclases.
- **Encapsulamiento, abstracción y polimorfismo**: mencionados como pilares adicionales del modelo.

Tipos de BDOO (según cátedra):

- **BDOO puras**: totalmente orientadas a objetos (ejemplos: ObjectDB, db4o).
- **Objeto-relacionales**: extienden bases SQL con objetos (ejemplo: PostgreSQL).
- **Librerías OO**: para lenguajes orientados a objetos, pero sin control transaccional completo.

La cátedra también menciona las **13 reglas fundamentales** de las bases de datos orientadas a objetos (síntesis de sus puntos): gestionar objetos complejos, identidad única, encapsulamiento, tipos y clases, jerarquías de clase, sobrecarga y polimorfismo, lenguaje computacionalmente completo, extensibilidad, persistencia, gestión de memoria, concurrencia, recuperación y consultas ad hoc.

5.2 Bases de datos multimedia

Almacenan y gestionan imágenes, audio, video, gráficos y animaciones. Según la cátedra, los **datos** se dividen en:

- **Estructurados**: los metadatos (autor, fecha, formato).
- **No estructurados**: el contenido multimedia en sí.

Los **tipos** de bases multimedia son:

- **Referenciales**: acceden por metadatos (autor, fecha, formato, etc.).
- **Descriptivas**: analizan el contenido multimedia (búsqueda por imagen, por video, etc.).

Desafíos: volumen de datos, acceso rápido, búsqueda por contenido, formatos diversos.
Aplicaciones: redes sociales, plataformas de video/audio, bancos de imágenes, educación digital.

5.3 Bases de datos biológicas: gestión de datos del genoma

Son repositorios de información científica como secuencias genéticas, proteínas y estudios clínicos, que facilitan la investigación en biología, genética y bioinformática. La cátedra distingue tres **tipos**:

- **Primarias**: datos crudos (ejemplos: GenBank, PDB).
- **Secundarias**: procesan y filtran las primarias (ejemplo: SwissProt).

- **Compuestas**: integran varias fuentes.

Gestión de datos de genoma y nuevas tecnologías: la cátedra destaca el uso de **Big Data e Inteligencia Artificial** para analizar grandes volúmenes de datos genómicos, lo que permite descubrir nuevos tratamientos, asociar genes a enfermedades y personalizar medicinas.

5.4 Bibliotecas digitales

Definidas por la cátedra como una *colección estructurada de documentos digitales y servicios interactivos*, que da acceso global, rápido y fácil a recursos académicos, culturales y científicos.

Características (según cátedra): metadatos para búsqueda eficiente, gestión de derechos de autor y licencias, y variedad de formatos (HTML, XML, PDF, ePub, etc.).

Ejemplo aplicado a la vida cotidiana: un servicio como Spotify combina varias de estas arquitecturas a la vez: usa una base **multimedia** para transmitir archivos de audio (contenido no estructurado) junto con sus metadatos —artista, álbum, duración— que funcionan como el acceso "referencial"; y organiza su catálogo con una lógica similar a una **biblioteca digital**, permitiendo búsquedas por género, artista o año gracias a esos metadatos bien estructurados.

UNIDAD 2: BASES DE DATOS CORPORATIVAS

Tema 1: Base de Datos Corporativas 🔵📘 (fuente: diapositivas de cátedra, Ing. Cozzolino)

1.1 Concepto, necesidades y campo de aplicación

¿Qué es una Base de Datos Corporativa? La cátedra la presenta directamente vinculada al concepto de **Data Warehouse**: un *repositorio centralizado que almacena toda la información proveniente de sistemas internos y externos*. Es un **sistema estructurado que integra y gestiona grandes volúmenes de información**, y la cátedra remarca un punto conceptual importante: **no es un producto, sino un conjunto de técnicas y procesos**.

¿Por qué implementar una Base de Datos Corporativa? (razones que da la cátedra):

- Integración y centralización de la información dispersa.
- Mejora en la calidad y coherencia de los datos.

- Optimización del acceso y la seguridad.
- Facilitación de la visualización y el análisis avanzado.
- Soporte para la toma de decisiones estratégicas.

Campo de aplicación del Data Warehouse (según cátedra):

- **Empresas multinacionales**: consolidación de operaciones globales.
- **Salud**: historiales médicos, gestión de recursos.
- **Educación**: registros académicos, procesos administrativos.
- **Gobierno**: registros civiles, control fiscal.
- **Retail y servicios**: manejo de inventario, clientes y ventas.

1.2 Componentes, fases y procesos vinculados

Componentes de un Data Warehouse (según cátedra):

- **Hardware**: infraestructura física — servidores, redes.
- **Software**: SGBD, herramientas ETL, middleware.
- **Datos**: información almacenada, estructurada.
- **Usuarios**: administradores, analistas, usuarios finales.
- **Procedimientos**: políticas de seguridad, backup, accesos.

El flujo completo (tal como lo esquematiza la cátedra) va desde la **infraestructura física** (servidores, sistemas de almacenamiento) y el **SGBD/herramientas ETL**, pasando por la **información almacenada** (estructurada y no estructurada) que se organiza en **bases de datos data marts**, hasta llegar a los **usuarios finales** (con distintos niveles de acceso) y a los **procedimientos** (políticas de seguridad, backups, recuperación, gestión de accesos y permisos, auditorías y mantenimiento), que en conjunto producen la **salida de uso de la información**: informes de negocio y análisis predictivo.

Fases del Desarrollo de un Data Warehouse (según cátedra) — este esquema es central y probablemente muy evaluable:

1. **Análisis de requerimientos**
    - Identificación de usuarios clave.
    - Necesidades de información (estratégica, táctica y operativa).
    - Fuentes de datos y frecuencia de actualización.

- Criterios de calidad de datos.

2. **Diseño conceptual y lógico**
   - Modelado dimensional (esquema estrella, copo de nieve).
   - Definición de métricas y dimensiones.
   - Integración de fuentes heterogéneas.
3. **Implementación**
   - Elección del SGBD y la plataforma.
   - Proceso ETL (Extracción, Transformación y Carga).
   - Carga de datos históricos.
   - (La cátedra lo resume visualmente como tres pasos: **Ingesta de Datos → Transformación de Datos → Visualización de Datos**.)
4. **Pruebas**
   - Validación de calidad de datos.
   - Verificación de integridad referencial.
   - Pruebas de rendimiento y carga.
5. **Capacitación de usuarios**
   - Formación en herramientas de consulta (ej. Power BI, Tableau).
   - Guía de uso y soporte inicial.
   - Feedback de los usuarios.
6. **Mantenimiento y optimización**
   - Automatización de procesos ETL.
   - Control de crecimiento del almacenamiento.
   - Optimización de consultas SQL.

Procesos vinculados (transversales a todas las fases): administración de usuarios y roles, respaldo y recuperación, monitoreo y ajuste de rendimiento, migración y actualización de datos.

Buenas prácticas que resalta la cátedra: planificación detallada, documentación clara y actualizada, participación de los usuarios finales, y evaluación constante del rendimiento —

resumidas en tres ejes: **Calidad, Usuarios, Métricas**.

Ejemplo aplicado a la vida cotidiana: una cadena de farmacias con 200 sucursales tiene una base transaccional (operativa) por cada punto de venta. Para que la gerencia general sepa cuál es el producto más vendido a nivel nacional este mes, no alcanza con mirar una sola sucursal: se necesita una Base de Datos Corporativa (Data Warehouse) que integre y centralice la información de las 200 tiendas — pasando por las seis fases que vimos: desde entender qué necesita saber cada gerente (requerimientos), diseñar cómo se van a organizar esos datos (esquema estrella), implementarlo con un proceso ETL, probarlo, capacitar a los gerentes en el uso del dashboard, y mantenerlo optimizado con el tiempo.

Tema 2: El proceso ETL 🔵📘 (fuente: diapositivas de cátedra, Ing. Cozzolino)

2.1 y 2.2 Introducción. El proceso de transformación de datos. Consideraciones

¿Qué es el Proceso ETL? Según la definición exacta de cátedra: es el *paso clave para la construcción de un Data Warehouse*, que *asegura calidad, consistencia y accesibilidad de la información*. **ETL = Extraer, Transformar y Cargar datos.**

Fase 1 — Extracción:

- Selección de fuentes: bases de datos, archivos planos, APIs.
- Limpieza y análisis inicial.
- Uso frecuente de SQL.

Fase 2 — Transformación:

- Normalización, validación, filtrado.
- Conversión de formatos, codificaciones, idiomas.
- Reglas específicas por negocio.

Fase 3 — Carga:

- Puede ser inicial o incremental.
- Flujo hacia un almacén central (cloud/local).
- Requiere monitoreo y control de errores.

Componentes del Proceso ETL que la cátedra destaca como requisitos transversales a las tres fases: compatibilidad y precisión, registro y auditoría, manejo de múltiples formatos, y tolerancia a fallas y escalabilidad.

Tipos de Tablas de Hechos

Este es un contenido específico que da la cátedra sobre el modelado del Data Warehouse (vinculado también al Tema 3, Data Warehousing):

- **Tabla Transaccional**: cada evento es una fila; las filas solo existen si una medida realmente ocurrió (ej. cada venta genera una fila); permiten mayor nivel de filtrado.
- **Tabla Snapshot Periódica**: sumariza varias medidas basado en un período de tiempo; cada fila muestra "el grano" en el tiempo, no la transacción individual; una fila se inserta incluso si no hubo transacciones (agrega la fila en cero) — por ejemplo, una foto del inventario de cada sucursal al cierre de cada mes.
- **Tabla Factless (sin métricas)**: sirven para analizar **lo que NO pasó**; las filas no tienen medidas numéricas, solo registran un conjunto de dimensiones en un punto del tiempo (ej. qué estudiante asistió a qué curso en qué fecha, sin ninguna "cantidad" asociada).

Dimensiones Especiales

Otro contenido puntual de cátedra, dentro del modelado dimensional del Data Warehouse:

- **Dimensión Calendario**: existe en todos los Data Warehouse; las tablas de hechos se suelen analizar a través del tiempo; su llave primaria es un número entero con el formato adecuado para representar la fecha correctamente.
- **Dimensión Role Playing**: una misma dimensión puede ser referenciada varias veces por la misma tabla de hechos, pero cumpliendo un rol diferente cada vez (por ejemplo, la dimensión "Fecha" usada como fecha de pedido, fecha de envío y fecha de entrega dentro de la misma tabla de hechos "Ventas").
- **Dimensión Junk**: combina en una sola tabla varias combinaciones de indicadores o flags que se pueden dar en el mundo real, para no tener que crear una dimensión separada por cada uno.
- **Dimensión Snowflake**: dentro de un Data Warehouse pueden darse relaciones jerárquicas; cuando una dimensión se normaliza, se obtienen dos (o más) tablas relacionadas por una llave foránea (es el mismo concepto de "esquema copo de nieve" que vimos en el modelado dimensional).

2.3 Herramientas de asistencia y automatización

Además de lo desarrollado en el Tema 1 de esta unidad (donde la cátedra ubica el ETL como uno de los componentes de la arquitectura de BI/Data Warehouse), existen herramientas específicas para automatizar y orquestar procesos ETL, como Talend, Pentaho Data Integration, Informatica PowerCenter, Microsoft SSIS y, en entornos de nube, servicios como Azure Data Factory o AWS Glue. *(Este listado de herramientas puntuales es complemento 🟡, ya que las diapositivas de cátedra no detallan marcas específicas en el Tema de ETL.)*

Ejemplo aplicado a la vida cotidiana: pensá en una tarjeta de crédito que te clasifica los gastos del mes por categoría (comida, transporte, entretenimiento). El proceso ETL nocturno **extrae** cada movimiento de la base transaccional, lo **transforma** (le asigna una categoría según el rubro del comercio, corrige el formato de fecha, valida que el monto sea coherente) y lo **carga** en el Data Warehouse. Ahí probablemente se guarde como una **tabla transaccional** (una fila por cada compra), mientras que el resumen de "saldo disponible al cierre de cada mes" que también ves en la app se arma con una **tabla snapshot periódica** — la misma distinción que hace la cátedra entre estos dos tipos de tabla de hechos.

Tema 3: Data Warehousing 🟡 (complementa el Tema 1 de esta unidad, donde ya está lo literal de cátedra)

> 📌 Nota: la Ing. Cozzolino desarrolla el **Data Warehouse** principalmente dentro del **Tema 1 — Bases de Datos Corporativas** (ver más arriba: qué es, componentes, y sobre todo las **6 fases de desarrollo**, que es contenido literal 🔵📘). Lo que sigue acá complementa ese contenido con la mirada específica del **modelado dimensional** (esquema estrella/copo de nieve/mixto) que el programa ubica en este Tema 3, usando conocimiento general del área 🟡, ya que no tengo las diapositivas específicas de este tema si es que tu profesora dio una diapositiva separada para él.

3.1 Introducción, requerimientos, diferencias con OLTP

Un **Data Warehouse (DW)** es un repositorio central de datos históricos, integrados desde múltiples fuentes, diseñado específicamente para el análisis y no para la operación diaria. Sus requerimientos son distintos a los de una base OLTP:

Característica	OLTP (operativa)	Data Warehouse (analítica)
Objetivo	Registrar transacciones	Analizar históricos
Datos	Actuales, muy normalizados	Históricos, desnormalizados
Consultas	Cortas, frecuentes	Largas, complejas, poco frecuentes

Usuarios	Empleados operativos	Analistas y directivos
Actualización	Constante (inserts/updates)	Por lotes (cargas periódicas)

3.2 Arquitectura: bloques de fuente de información, construcción de DW y Mercado de Datos, acceso y uso, administración de datos y metadatos, transporte e infraestructura

La arquitectura típica de un DW se organiza en capas o "bloques":

- **Capa de fuentes**: los sistemas operativos que originan los datos.
- **Capa de construcción/staging**: área intermedia donde ocurre el proceso ETL antes de cargar al DW definitivo.
- **Capa de almacenamiento del DW y Data Marts**: el repositorio central y sus subconjuntos especializados por área.
- **Capa de acceso y uso**: herramientas de reportes, cubos OLAP y dashboards que consumen el DW.
- **Capa de administración de datos y metadatos**: gestiona la calidad, seguridad y el diccionario de datos.
- **Capa de transporte e infraestructura**: la red, servidores y mecanismos que mueven los datos entre capas.

3.3 Construcción: ciclo de desarrollo. Modelado: esquema estrella, copo de nieve y mixto. Metadatos. Data Marts

El **ciclo de desarrollo** de un DW sigue fases similares a cualquier proyecto de software: Planeación → Requerimientos → Análisis y Diseño → Integración y Pruebas → Despliegue.

Modelado dimensional:

- **Esquema estrella**: una tabla de hechos central (por ejemplo, "Ventas", con métricas numéricas como monto y cantidad) rodeada de tablas de dimensión desnormalizadas (Producto, Cliente, Tiempo, Sucursal). Es el modelo más simple y rápido de consultar.
- **Esquema copo de nieve (snowflake)**: igual que el estrella, pero las dimensiones se normalizan en sub-tablas adicionales (por ejemplo, Producto se separa en Producto → Categoría → Departamento). Ahorra espacio pero las consultas requieren más *joins*.
- **Esquema mixto**: combina ambos enfoques según convenga: algunas dimensiones desnormalizadas (estrella) y otras normalizadas (copo de nieve) dentro del mismo

modelo.

Metadatos y diccionario de datos: los metadatos documentan el significado, origen y reglas de transformación de cada campo del DW. El diccionario de datos es el catálogo formal donde se detalla cada tabla y columna, fundamental para que los analistas entiendan qué están consultando.

Data Marts: son subconjuntos del Data Warehouse, enfocados en un área específica del negocio (por ejemplo, un Data Mart solo de Ventas, o solo de RRHH), más pequeños y rápidos de consultar que el DW completo.

Ejemplo aplicado a la vida cotidiana: pensá en el Data Warehouse de una plataforma de streaming como Netflix. Tiene una tabla de hechos "Reproducciones" (qué usuario vio qué contenido, cuánto tiempo, en qué dispositivo) rodeada de dimensiones: Usuario, Contenido, Fecha, Dispositivo. Con un esquema estrella, Netflix puede responder rápidamente preguntas como "¿cuántas horas se vieron de una serie el fin de semana pasado, por país?", combinando la tabla de hechos con las dimensiones de Fecha y Usuario.

Tema 4: Procesamiento Analítico 🟡

4.1 Introducción y características

El procesamiento analítico busca responder preguntas de negocio explorando los datos desde múltiples ángulos simultáneamente (por producto, por región, por tiempo), a diferencia de una consulta SQL tradicional que responde una pregunta puntual a la vez.

4.2 Análisis multidimensional. OLAP: definición, arquitectura, reglas, evaluación, modelos MOLAP, ROLAP, HOLAP. Cubos

OLAP (Online Analytical Processing) es la tecnología que permite analizar datos organizados como un **cubo multidimensional**: cada "eje" del cubo es una dimensión (Producto, Tiempo, Región) y cada celda contiene una métrica (por ejemplo, Ventas). Esto permite operaciones características:

- **Drill-down**: bajar de nivel de detalle (de "año" a "mes" a "día").
- **Roll-up**: subir de nivel (de "ciudad" a "país").
- **Slice**: fijar un valor de una dimensión y ver el resto (ej. "solo el producto X").
- **Dice**: filtrar por varios valores de varias dimensiones a la vez.
- **Pivot**: rotar el cubo para cambiar la perspectiva de análisis.

Modelos de implementación:

- **MOLAP (Multidimensional OLAP)**: el cubo se pre-calcula y almacena físicamente en una estructura multidimensional propia. Consultas muy rápidas, pero requiere más espacio y tiempo de procesamiento previo.
- **ROLAP (Relational OLAP)**: el cubo se calcula "al vuelo" con consultas SQL sobre el esquema estrella/copo de nieve relacional. No requiere pre-cálculo, pero las consultas son más lentas.
- **HOLAP (Hybrid OLAP)**: combina ambos: mantiene el detalle en modo relacional (ROLAP) y pre-calcula los resúmenes más consultados en modo multidimensional (MOLAP).

Ejemplo aplicado a la vida cotidiana: un gerente de una cadena de cines abre un tablero OLAP y ve las ventas totales de entradas por país (roll-up). Le llama la atención un número bajo, así que hace *drill-down* hasta ver las ventas por cine específico, y luego por película y horario, hasta descubrir que una sala en particular tuvo mala venta por un problema con la proyección de una función. Todo esto lo hace explorando el mismo cubo de datos, sin escribir una sola consulta SQL nueva.

Tema 5: Minería de Datos 🟡

5.1 Introducción y definición, usuarios

La **minería de datos (Data Mining)** es el proceso de descubrir patrones, relaciones y conocimiento no evidente dentro de grandes volúmenes de datos históricos, usando técnicas estadísticas y de aprendizaje automático. A diferencia del OLAP (que responde preguntas que el usuario ya sabe hacer), la minería de datos busca **patrones que nadie preguntó explícitamente** (por ejemplo, "¿qué productos se compran juntos sin que nadie lo haya notado?").

Los usuarios típicos son analistas de negocio, científicos de datos y áreas de marketing, riesgo crediticio o fraude.

5.2 Herramientas: análisis estadístico, descubrimiento de conocimientos, otras técnicas. Ejemplos de aplicación

Las técnicas principales incluyen:

- **Análisis estadístico**: regresión, correlación, análisis de varianza, para cuantificar relaciones entre variables.

- **Descubrimiento de conocimiento (KDD)**: proceso completo que incluye selección, limpieza, transformación, minería propiamente dicha e interpretación de resultados.
- **Clustering (segmentación)**: agrupar registros similares sin categorías predefinidas (ej. segmentar clientes por comportamiento de compra).
- **Reglas de asociación**: descubrir qué elementos aparecen juntos frecuentemente (el clásico "canasta de mercado").
- **Árboles de decisión y clasificación**: predecir una categoría (ej. "este cliente va a dejar de pagar" sí/no).

Ejemplo aplicado a la vida cotidiana: cuando Mercado Libre o Amazon te muestran "los clientes que compraron esto también compraron...", ese resultado no lo escribió a mano ningún empleado: salió de un algoritmo de minería de datos (reglas de asociación) que analizó millones de compras históricas y detectó que, por ejemplo, quien compra una funda de celular suele comprar también un vidrio templado.

UNIDAD 3: BASES DE DATOS NOSQL

Tema 1: Concepto de NoSQL 🔵📘 (fuente: diapositivas de cátedra, Prof. Ing. Fernando Zapata — *"Intro DB NoSQL"*)

Repaso del modelo relacional y transacciones ACID

Antes de entrar a NoSQL, la cátedra repasa el **modelo relacional**: estructura basada en entidades/tablas y relaciones (claves primarias, foráneas), con restricciones de integridad (integridad referencial, estructural, etc.) que llevan a la **normalización**. Esto optimiza el espacio de almacenamiento, pero tiene un costo de procesamiento (armado y desarmado de la información al consultar).

Las **transacciones en el modelo relacional** siguen las propiedades **ACID**:

- **Atomicidad**: la transacción es "todo o nada"; si una parte falla, falla la transacción completa.
- **Consistencia**: solo se escriben datos válidos; si una transacción viola las reglas de integridad, se revierte y la base vuelve a un estado consistente.
- **Aislamiento**: transacciones simultáneas no se afectan entre sí; el sistema ejecuta una por completo antes que la otra (o viceversa), sin interferencias.

- **Durabilidad**: una vez confirmada (commit) una transacción, no se pierde, aunque el sistema falle después (se garantiza con backups y logs de transacciones).

Punto clave que remarca la cátedra: no todos los modelos de bases de datos están basados en ACID. Las bases de datos **NoSQL no proporcionan todos los conceptos ACID**, ya que su objetivo es responder con la máxima velocidad, asumiendo que se puede tolerar alguna pérdida de datos. Por eso, para las bases NoSQL se definió el **teorema CAP** (Consistencia, Disponibilidad, Tolerancia a Particiones — un sistema distribuido solo puede garantizar dos de las tres al mismo tiempo).

1.1 a 1.4 Las cuatro familias de bases de datos NoSQL

La cátedra confirma que **NoSQL significa "Not Only SQL"** y desarrolla las mismas cuatro familias con ejemplos concretos de motores:

- **Documentales**: almacenan la información como un documento, generalmente en JSON o XML, con una clave única por registro. Permiten búsquedas por clave-valor y también consultas más avanzadas sobre el contenido. Son **las más versátiles** — se usan en gran cantidad de proyectos, incluso muchos que tradicionalmente irían sobre relacional. Ejemplos: **MongoDB**, **CouchDB**.
- **Clave-Valor**: el modelo NoSQL más popular y el más simple. Cada elemento tiene una llave única que permite recuperar la información muy rápido; suele almacenarse como un objeto binario (BLOB). Muy eficientes en lectura y escritura. Ejemplos: **Cassandra**, **BigTable**, **HBase**.
- **En Grafo**: la información se representa como nodos (entidades) y aristas (relaciones), aprovechando la teoría de grafos para recorrerla. Para un rendimiento óptimo, la estructura debe estar totalmente normalizada. Ofrece navegación más eficiente entre relaciones que el modelo relacional. Ejemplos: **Neo4j**, **InfoGrid**, **Virtuoso**.
- **Orientadas a Objetos**: la información se representa como objetos, igual que en los lenguajes de programación orientados a objetos (Java, C#, Visual Basic .NET). Ejemplos: **Zope**, **Gemstone**, **Db4o**.

El mundo de los motores NoSQL (motores específicos que detalla la cátedra)

- **Riak**: almacén de datos clave-valor NoSQL distribuido, con alta disponibilidad, tolerancia a fallas, simplicidad operativa y escalabilidad. Pasó a ser completamente código abierto en agosto de 2017.
- **Redis** (2009): motor clave-valor **en memoria**, basado en tablas de hash, que opcionalmente puede usarse como base de datos durable/persistente. Escrito en ANSI C por Salvatore Sanfilippo (patrocinado por Redis Labs). Twitter lo usa para: *caching y*

session storage, streaming & messaging, real-time data store.

- **Cassandra** (Apache, 2008): NoSQL distribuido, basado en un modelo de almacenamiento "tabular" de clave-valor, código abierto, escrito en Java. Permite grandes volúmenes de datos en forma distribuida. Lo usan **Twitter**, **Apple** y **Netflix**.
- **Apache HBase**: sistema gestor de base de datos distribuida no relacional, de código abierto, **orientada a columnas**, modelada a partir de Google BigTable, escrita en Java. Parte del proyecto Hadoop, se ejecuta sobre HDFS.
- **Apache CouchDB** (2005): gestor de bases de datos **documental**, de código abierto, enfocada en la facilidad de uso — "una base de datos que asume la web de manera completa". Usa JSON y JavaScript.

Tabla comparativa de motores NoSQL (según cátedra)

	Documental	**Columnar**	**Clave-valor (mem)**	**Documental indexado**
Data model	Documentos BSON/JSON	Tablas con filas y columnas	Clave-valor, listas, sets, hashes	Documentos JSON indexados
Usos	Apps web, sistemas flexibles	Big Data, IoT, análisis de logs	Caché, colas, sesiones de usuario	Búsquedas y análisis (tiempo real)
Escalabilidad	Horizontal (Sharding)	Horizontal (altamente)	Escalable, limitado por RAM	Escalable en clusters
Persistencia	Sí	Sí	Opcional	Sí
Velocidad	Alta	Alta	Muy alta	Alta (search)
Licencia	SSPL / Apache 2.0	Apache 2.0	BSD	Apache 2.0 / Elastic License
Empresas usuarias	IBM, Walmart, Deloitte	Apple, Netflix, Spotify	GitHub, Twitter, Snapchat	Uber, LinkedIn, eBay

Ventajas de NoSQL que resalta la cátedra: se ejecutan en máquinas con pocos recursos y bajo costo; ofrecen **escalabilidad horizontal** (mejoran el rendimiento añadiendo más nodos); pueden manejar grandes cantidades de datos con estructura distribuida (muchas veces mediante tablas hash); no generan cuellos de botella.

Ejemplo aplicado a la vida cotidiana: cuando Twitter/X te muestra el timeline al instante, no está usando una sola base de datos: probablemente usa **Redis** como caché para lo que ves apenas abrís la app (súper rápido, en memoria), **Cassandra** para almacenar el enorme volumen histórico de tweets de forma distribuida, y quizás una base en **grafo** para calcular a quién seguís y quién te sigue. Cada motor NoSQL resuelve una parte distinta del problema, sacrificando algo de las garantías ACID a cambio de velocidad y escala — la misma idea del teorema CAP que menciona la cátedra.

Tema 2: Bases de datos documentales — MongoDB 🔵📘 (fuente: diapositivas de cátedra: *"Intro MongoDB"*, *"Modelado"*, *"Índices y rendimiento"*, *"Operación — Gestión de Datos"*)

2.1 Comparación con el modelo relacional. Historia y definición

Historia de MongoDB (según cátedra):

- **2007**: comienza como proyecto de desarrollo de **10gen Inc.** (hoy MongoDB Inc.), cuando la empresa desarrollaba una plataforma como servicio (PaaS) similar a Google App Engine.
- **2009**: se lanza como **producto independiente**, bajo licencia Affero AGPL.
- **2011**: con la versión 1.4, se considera **lista para uso en producción**.

Definición de cátedra: *MongoDB es una Base de Datos NoSQL, de propósito general, escrita en C++, que puede manejar grandes volúmenes de datos (de ahí "humongous"), de fácil escalado horizontal, y schemaless (no orientado a esquema — no requiere una estructura fija de datos)*.

Es usada por empresas como **Coinbase, eBay, Google, The Weather Channel, Bosch**, entre miles más.

Licenciamiento (evolución que detalla la cátedra):

- Originalmente: **AGPL** (Affero GPL de GNU) — fuerte copyleft, con cobertura de servicios de red y SaaS.
- **Desde 2018**: **SSPL** (Server Side Public License) — garantiza la apertura del código si MongoDB se ofrece como servicio en la nube.
- Los **drivers** para los distintos lenguajes de programación usan licencia **Apache 2.0**.

- MongoDB Inc. también ofrece una **licencia comercial** con características avanzadas: integración con SASL, LDAP, Kerberos, SNMP, búsqueda de texto avanzada, herramientas de gestión, monitoreo, backup y soporte.

¿Cuándo aplica MongoDB? (criterios exactos de cátedra) — para soluciones que:

- requieran escalabilidad horizontal o replicación;
- utilicen datos semiestructurados o dinámicos;
- necesiten alto rendimiento en grandes volúmenes;
- **no** requieran procesos transaccionales complejos;
- cuyos modelos no se basen en relaciones.

2.2 Conceptos de Base de Datos, Colección y Documento

MongoDB es una **base de datos orientada a documentos**: los objetos de JavaScript (JSON) se guardan como **BSON** (representación binaria de JSON). Un documento es un conjunto de pares **clave-valor**:

```
{
    "nombre" : "Juan",
    "apellido" : "Perez",
    "edad" : 20
}
```

La jerarquía equivalente entre MongoDB y una base relacional (ya vista en el resumen anterior) sigue siendo válida: Server↔Servidor, Database↔Base de Datos, Collection↔Tabla, Document↔Fila, Fields↔Columna.

Ecosistema relacionado — MEAN Stack y Mongoose: la cátedra menciona que MongoDB suele combinarse con **Express.js** (framework web para Node.js), **Angular** (framework de Google en TypeScript) y **Node.js** (entorno de ejecución JS del lado del servidor, asíncrono, basado en el motor V8 de Google) formando el **stack MEAN**. Además, **Mongoose.js** es una biblioteca JavaScript que permite definir esquemas con datos fuertemente tipados sobre MongoDB — funciona como un **ODM** (Object Document Mapper, equivalente a un ORM mapeando documentos en lugar de tablas).

Tabla SQL vs. MongoDB (ya vista, sigue vigente)

SQL	MongoDB
`SELECT * FROM users`	`db.users.find()`

`SELECT * FROM users WHERE status = "A"`	`db.users.find({ status: "A" })`
`SELECT * FROM users WHERE age > 25`	`db.users.find({ age: { $gt: 25 } })`
`SELECT * FROM users ORDER BY user_id ASC`	`db.users.find().sort({ user_id: 1 })`
`SELECT COUNT(*) FROM users`	`db.users.count()`

Tipos de Datos en MongoDB (contenido específico y detallado de cátedra)

Los **tipos de JSON** son limitados: `null`, `boolean`, `numeric`, `string`, `array`, `object` — y **no distinguen** entre float/int/32bits/64bits, **no tienen formato de fecha**, y **no manejan expresiones regulares**.

MongoDB extiende esos tipos (lista exacta de cátedra, con sintaxis):

Tipo	Sintaxis de ejemplo
null	`{"x" : null}`
boolean	`{"x" : true}`
number	`{"x" : 3.14}` / `{"x" : NumberInt("3")}` (4 bytes) / `{"x" : NumberLong("3")}` (8 bytes)
string (UTF-8)	`{"x" : "foobar"}`
date	`{"x" : new Date()}`
regular expression	`{"x" : /foobar/i}`
array	`{"x" : ["a", "b", "c"]}`
embedded docs	`{"x" : {"foo" : "bar"}}`
object id (12 bytes)	`{"x" : ObjectId()}`
binary data	(el único tipo que no usa UTF-8)
code	`{"x" : function() { /* ... */ }}`

Documentos embebidos — ejemplo de cátedra:

```
db.contacto.insert({
    "nombre" : "Ana",
    "apellido" : "Sanchez",
    "domicilio" : {
        "calle" : "San Juan",
        "num" : 123,
        "ciudad" : "Mendoza"
    }
})
```

_id y ObjectId: la clave `_id` identifica de forma única a cada documento en la colección. `ObjectId` es el tipo de dato por defecto para `_id`, diseñado para ser un identificador global en sistemas distribuidos, de **12 bytes**. Como inicia con un timestamp, permite deducir la fecha y hora de creación del documento (y da un orden inicial por fecha de creación). El usuario puede especificar el `_id` manualmente o dejar que MongoDB lo genere.

Operaciones básicas (CRUD) — sintaxis exacta de cátedra

```
use agenda                          // selecciona (o crea) la base de datos activ
db                                  // muestra en qué base estoy
show dbs                            // muestra las bases existentes
db.createCollection("contactos")    // crea una colección explícitamente
show collections                    // muestra las colecciones de la base activa

db.contactos.insertOne({ "nombre":"Juan", "apellido":"Perez" })  // si la colecci
db.contactos.insertMany([ {"_id":0}, {"_id":1}, {"_id":2} ])     // carga masiva

db.contactos.find().pretty()                                     // consulta con
db.contacto.find({ edad: 40, nombre: "Juan" })                   // condición exa
db.contacto.find({ edad: { "$gte":18, "$lte":30 } })              // rango de val

db.contacto.updateOne({"nombre":"Juan"}, { $set: {'edad':40} })  // modificar
db.contacto.deleteOne({ "nombre":"Juan" })                        // eliminar un
db.contacto.remove({"nombre":"Juan"})                             // elimina TODO

db.contactos.drop()                 // elimina la colección completa
db.dropDatabase()                   // elimina la base de datos completa
```

Operadores de actualización sobre arrays (según cátedra):

- `$push` → agrega elementos al final del arreglo (ej. `{$push: {tags:"música"}}`);

combinado con `$each` permite insertar múltiples valores de una vez.

- `$pop` → elimina elementos desde un extremo (pila o cola): `{$pop:{tags:1}}` elimina el último, `{$pop:{tags:-1}}` elimina el primero.
- `$pull` → elimina todos los elementos del array que cumplan una condición dada.
- Otros operadores mencionados: `$unset` (elimina la clave), `$inc` (incrementa el valor), `$in` (verifica si el valor está en una lista).

Relaciones en MongoDB (Cardinalidad) — contenido específico de cátedra

MongoDB modela relaciones de **Uno a Uno**, **Uno a Muchos** y **Muchos a Muchos**, vinculando documentos de distintas colecciones (a diferencia del modelo relacional, acá no hay claves foráneas obligatorias con integridad referencial automática).

Ejemplo de Uno a Muchos (cátedra usa "Carrera" y "Facultad"):

```
Carrera:   { _id: 3, nombre: "Abogacía", facultad_Id: 2 }
Facultad:  { _id: 2, nombre: "Derecho" }
```

Ejemplo de Muchos a Muchos (cátedra usa "Usuario" y "Grupo", con arrays de referencias cruzadas):

```
Usuario: { _id: 1, apellido: "Perez", nombre: "Juan", grupos: [1, 3, 7] }
Grupo:   { _id: 3, nombre: "Andinismo", usuario: [1, 3, 9, 15] }
```

Consultas avanzadas: el framework de Agregación (**`aggregate`**)

El método **`aggregate`** toma una lista de "etapas" (stages) que se ejecutan en secuencia para filtrar, transformar y agrupar documentos:

- **`$match`** : filtra los documentos que cumplen una condición.
- **`$group`** : agrupa documentos y permite operaciones como `$sum` (ej. `_id: null` agrupa TODOS los documentos).
- **`$project`** : remodela la salida — incluye/excluye campos, renombra claves, calcula nuevos valores (ej. con `$add`), o aplica expresiones condicionales (`$cond` , con `if/then/else` , por ejemplo clasificar "Adulto" vs "Menor" según la edad).
- **`$lookup`** : permite un **"join"** entre dos colecciones relacionadas (ej. `clientes` y `pedidos`), especificando `from` , `localField` , `foreignField` y el alias `as` para el resultado combinado.

Ejemplo de cátedra:

```
db.miColeccion.aggregate([
   { $match: { condicion: "alguna_condicion" } },
   { $group: { _id: null, suma: { $sum: "$valor" } } }
]);
```

Colecciones de Series Temporales (Time Series — MongoDB 5.0+)

Mecanismo optimizado (a partir de MongoDB 5.0) para gestionar datos que cambian en el tiempo, donde cada documento representa una medición en función del tiempo, agrupada internamente en **buckets**. Se crean con:

```
db.createCollection("log_temp", {
  timeseries: { timeField:"timestamp", metaField:"sensor_info", granularity:"minu
})
```

Ventajas (según cátedra): compresión automática de datos, escrituras y lecturas optimizadas por tiempo. Casos de uso típicos: **IoT, sensado, logs**.

¿Cuándo usarlas? cuando se insertan muchos datos ordenados por tiempo, se consulta por rangos de tiempo, no se necesita modificar ni borrar datos viejos, y se requiere menor uso de disco y mayor rendimiento.

2.3 Consideraciones de rendimiento y optimización. Indización (fuente: *"Índices y rendimiento"*)

¿Cómo funciona un índice? Evita el *full-scan* (lectura secuencial de toda la colección). MongoDB usa un **B-Tree modificado** (más flexible que el de una base relacional), y busca por **clave-valor**, a diferencia de las columnas indexadas en una base relacional. Por defecto, el campo `_id` ya está indexado — por eso buscar por `_id` (ObjectId) es casi inmediato, mientras que buscar por un campo no indexado (ej. `usuario`) sobre millones de documentos genera demora.

Creando y eliminando índices:

```
db.contactos.createIndex({ apellido: 1 }, { name: "consulta_de_apellidos" })
db.contactos.dropIndex("consulta_de_apellidos")
db.users.getIndexes()
```

(El nombre por defecto de un índice, si no se especifica, es `campo_1`; la opción `{background:true}` evita bloquear el acceso a los datos de la colección mientras se crea el índice.)

Tipos de índices que menciona la cátedra: índices compuestos, índices *multikey* (sobre arrays), índices **geoespaciales**, e índices *hashed* (usados para sharding).

Medición de rendimiento: `db.col.find().explain("executionStats")` — permite ver `executionTimeMillis`, es decir, el costo real de ejecución de una consulta, con y sin índice.

Criterios para diseñar la estrategia de indización (checklist de cátedra — hay que analizar el impacto en cada uno de estos puntos): consultas (los índices las aceleran mucho), inserciones (los índices las hacen más lentas), eliminaciones, actualizaciones, movimiento de datos, y uso de memoria/disco. En definitiva: hay que **analizar efectividad y eficiencia**, ya que indexar todo no es gratis.

2.4 Modelado de datos en MongoDB (fuente: *"MongoDB Modelado"*)

La cátedra plantea el **proceso de diseño** en dos vías paralelas que confluyen: por un lado el **Modelo de Datos** (a partir de un dominio determinado: casos de uso, diagrama lógico, diagrama físico → modelo normalizado); por otro, el **Diseño de la Aplicación** (navegabilidad, interfaces) que genera **cambios necesarios** sobre ese modelo, llevando a un **modelo desnormalizado**. En MongoDB, este proceso de ida y vuelta entre modelo de datos y diseño de aplicación es explícito y central: **el modelo de datos en MongoDB se diseña en función de cómo la aplicación va a consultarlo**, mucho más que en el mundo relacional.

Dónde no usar MongoDB (recordatorio ya visto, confirmado por cátedra): no existen transacciones complejas multi-documento equivalentes a un RDBMS, y no existen JOINs nativos (se resuelven con `$lookup` en el framework de agregación, o embebiendo datos). Cada documento tiene un límite de **16 MB**; para archivos más grandes se usa la extensión **GridFS**.

Ejemplo aplicado a la vida cotidiana: pensá en la app de un gimnasio que registra las clases a las que se anota cada socio. Un socio (`Usuario`) tiene un array de clases (`grupos: [1,3,7]`) y cada clase (`Grupo`) tiene un array de socios anotados — exactamente el modelo Muchos a Muchos que muestra la cátedra con "Usuario" y "Grupo". Si el gimnasio quiere saber cuántos socios activos tiene por clase, usa `aggregate` con `$group` y `$sum`; si además quiere cruzar esa info con la colección de "pagos" para ver quién debe la cuota, usa `$lookup` — el "join" de MongoDB.

Tema 3: Plataformas y sistemas de soporte 🔵📘 (fuente: diapositivas de cátedra: *"Instalación y Configuración"*, *"Cloud"*, *"Backup"*)

3.1 Instalación y configuración del motor

Instalación de MongoDB Server (pasos de cátedra): descarga desde www.mongodb.com → instalación (como servicio o no) → funcionamiento → ejecución. Existen 2 ejecutables clave en la carpeta `bin`: **`mongod.exe`** (el servidor) y **`mongo.exe`** / **mongosh** (el cliente/shell).

En **Windows**, si no se instala como servicio, hay que crear manualmente el directorio de almacenamiento (`c:\data\db`) donde se guardan los **data files**, **index files** y **journal files** de cada colección.

El motor de almacenamiento: WiredTiger. MongoDB usaba originalmente el motor **MMAPv1** (hoy obsoleto) y pasó a usar **WiredTiger**, una plataforma NoSQL de gestión de datos con estas características (según cátedra):

- **Extensible**: soporta complementos (extensiones/librerías incorporadas).
- **Open Source** (licencia GNU GPL).
- Usa arquitectura **MVCC** (*MultiVersion Concurrency Control*): las actualizaciones no borran ni sobreescriben el dato viejo inmediatamente, sino que lo marcan como obsoleto y agregan el nuevo, manteniendo temporalmente varias versiones del mismo dato (solo la más reciente es válida). Esto evita que el gestor pierda tiempo "rellenando huecos" en memoria/disco al costo de tener que limpiar periódicamente los objetos obsoletos. En una base documental, esta estrategia permite escribir documentos en memoria contigua, evitando fragmentación.
- Permite **checkpointing** y **compresión** (usa compresión simple por defecto, de bajo costo).

El Journal (Write-Ahead Log): cada vez que WiredTiger recibe una operación de escritura en disco, primero la registra en una bitácora llamada *write-ahead log* (MongoDB lo llama **journaling**). Si el sistema se cae, toda escritura aún no persistida puede reejecutarse desde ese log. Existen **modos de durabilidad** (*durability modes*) que permiten elegir el balance entre rendimiento y confiabilidad:

- **FullSync**: confirma la escritura solo cuando los datos están completamente sincronizados en disco → máxima durabilidad, pero más lento.
- **NoSync**: no espera la sincronización en disco → mejor rendimiento, pero riesgo de pérdida de datos ante un fallo.
- **Write Concern**: define cuántos nodos deben confirmar una escritura antes de que MongoDB le responda al cliente que la operación tuvo éxito (puede exigir solo el nodo principal, esperar el journal, o exigir réplica en la mayoría del conjunto de réplicas) —

permite balancear velocidad vs. seguridad.

3.2 Implementación en cloud: MongoDB Atlas

La cátedra presenta dos grandes **alternativas** para implementar sistemas MongoDB basados en la nube:

1. **MongoDB Atlas** (Atlas UI / CLI): clusters y replicación en la nube, con escalabilidad, configuración sencilla y herramientas de administración incorporadas — pensado para simplicidad y rapidez de puesta en marcha.
2. **MongoDB Enterprise Server** (con **Ops Manager**): pensado para necesidades más avanzadas — integración con **Kubernetes**, alternativas de seguridad avanzada, e integración con herramientas de BI.

El flujo típico en Atlas involucra: creación de clusters/réplicas, gestión del **acceso** (usuarios, IPs permitidas), la **administración** del cluster, y la **conexión** de la aplicación (string de conexión).

Utilidades de migración y resguardo (Backup)

- **`mongoexport`** : aplicación **independiente** (no requiere el shell de mongo) que permite una exportación básica desde la misma máquina que ejecuta el servidor, generando un archivo de texto en formato JSON. Sintaxis: `mongoexport -d base -c colecc -o archivo_salida` (ej. `mongoexport -d facultad -c alumno -o alumnos.json`). Permite especificar path, usuario y contraseña.
- **`mongodump`** : permite realizar una **descarga masiva** de las bases de datos alojadas en el servidor, generando archivos `.bson` (datos binarios) y `.metadata.json` (información de índices). Los `.bson` pueden inspeccionarse con la utilidad `hexdump` o con `bsondump` .
- **`mongorestore`** : permite restaurar un backup generado con `mongodump` .
- **`mongoimport`** : permite importar contenido previamente exportado con `mongoexport` .

Backup en MongoDB Atlas — dos alternativas que ofrece Atlas (según cátedra):

- **Continuous Backups**: genera backups incrementales continuos, con recuperación a pocos segundos de la ejecución, y posibilidad de recuperación en **cualquier punto en el tiempo** dentro de las últimas 24 horas.
- **Cloud Provider Snapshots**: backups basados en la funcionalidad nativa del proveedor de servicios de clustering (ej. snapshots del propio AWS/Azure/GCP).

Ejemplo aplicado a la vida cotidiana: pensá en cómo un negocio de e-commerce chico podría dar sus primeros pasos con MongoDB: al principio prueba todo instalado en su

propia notebook (WiredTiger local, con journaling activado para no perder datos si se cuelga el sistema mientras carga productos). Cuando lanza a producción, migra a **MongoDB Atlas** para no tener que administrar servidores propios, contratando **Continuous Backups** para poder "volver en el tiempo" si algo sale mal con una migración de datos — y usa `mongodump` / `mongorestore` para bajarse una copia local de la base y probar cambios sin tocar la base real.

Cierre: cómo se relacionan las tres unidades

- **Unidad 1** te da el panorama: existen muchas arquitecturas de datos modernas (nube, móvil, geoespacial, BI, orientada a objetos, multimedia, biológica) y cada una responde a una necesidad distinta.
- **Unidad 2** profundiza en el caso específico de **consolidar datos de toda una empresa** para la toma de decisiones: cómo se extraen y transforman los datos (ETL), cómo se modelan para análisis (Data Warehouse, esquema estrella), cómo se exploran interactivamente (OLAP) y cómo se descubren patrones ocultos (minería de datos).
- **Unidad 3** profundiza en el caso específico de los **datos semiestructurados y la escalabilidad horizontal**, usando MongoDB como caso de estudio central de las bases documentales, con aplicación directa a GIS mediante GeoJSON.

Estas tres unidades, juntas, te dan el criterio para responder la pregunta central de la materia: frente a un requerimiento de negocio, **¿qué tecnología de gestión de datos es la más adecuada?** — no existe una respuesta única, sino la que mejor se ajusta al volumen, la estructura de los datos y el tipo de consulta que se necesita resolver.